API Documentation

Publication-quality reference for the Travel Without Barriers REST API, deployment model, authentication flow, and MCP integration.

Module: COMP3011 Web Services and Web Data

System stack: Next.js (frontend) • FastAPI (backend) • SQLite (data layer) • Railway/Vercel deployment • MCP integration

This document is designed to function both as a technical reference and as a user-oriented guide. It explains the system context, base conventions, authentication workflow, every REST endpoint currently exposed by the service, example requests and responses, common error cases, deployment notes, and the Model Context Protocol layer added to extend the API for AI-driven tooling.

The documentation reflects the implemented API shape used in the deployed project and is intended to make the capabilities, constraints, and expected payloads of the Travel Without Barriers platform immediately clear to markers, developers, and reviewers.

Contents

- 1. Introduction and documentation scope
- 2. System context and architecture
- 3. API conventions and base information
- 4. Authentication model and security behaviour
- 5. Resource model summary
- 6. Endpoint reference
- 6.1 Root endpoint
- 6.2 Authentication endpoints
- 6.3 Destination endpoints
- 6.4 Recommendation endpoint
- 6.5 Wishlist endpoints
- 7. Error handling and status codes
- 8. End-to-end usage workflows
- 9. Deployment, environments, and operational notes
- 10. MCP integration appendix
- 11. Local development and testing notes

1. Introduction and documentation scope

Travel Without Barriers (TWB) is a barrier-aware travel recommendation platform designed to help users identify destinations that are easier to plan, financially more manageable, and less overwhelming. The platform addresses a clear user need: many people want to travel but are discouraged by cost, crowd levels, and the complexity of choosing destinations that fit their circumstances.

The backend service is implemented as a RESTful API using FastAPI. It stores a curated tourism dataset in SQLite, computes derived recommendation metrics, supports user registration and login, and enables authenticated users to create and manage wishlists of destinations. A deployed Next.js frontend consumes the API, while a separate MCP server exposes selected backend capabilities as AI-callable tools.

This documentation is intentionally detailed. In addition to endpoint reference material, it explains how the system is structured, how the authentication flow works, what each resource represents, what errors clients should expect, and how external clients can integrate with the API safely and successfully.

Documentation goals

- Provide a complete and readable reference for every REST endpoint exposed by the API.
- Explain request format, authentication requirements, path parameters, query parameters, request bodies, response bodies, and expected error cases.
- Describe the deployed architecture and the relationship between the web frontend, backend API, database, and MCP layer.
- Support reproducibility by documenting local run commands, payload conventions, and common workflows.
- Demonstrate publication-quality API communication through clear structure, professional styling, and practical examples.

2. System context and architecture

The Travel Without Barriers system follows a layered architecture. At the presentation layer, the user interacts with a deployed Next.js application hosted on Vercel. The frontend is responsible for rendering recommendation cards, filters, and wishlist interfaces, but it does not implement business logic directly. Instead, it communicates with the backend service using HTTP requests.

The service layer is implemented using FastAPI and deployed on Railway. This layer handles request validation, authentication, routing, score calculation, relational operations over wishlists and items, and JSON serialization. The data layer is a SQLite database containing destination records, user accounts, wishlists, and wishlist items. The application also includes a Python-based MCP server that calls the deployed backend over HTTP, enabling AI-driven clients to interact with the same live system as the frontend.

Architectural components

- Frontend: Next.js application deployed on Vercel.
- Backend: FastAPI application deployed on Railway and served by Uvicorn.
- Database: SQLite database containing the tourism dataset and user-generated data.
- Data ingestion: a Python import script populates the destinations table from the source CSV dataset.
- MCP integration: a local MCP server uses HTTP calls to the deployed backend so that MCP-driven operations remain aligned with the live website.

High-level request flow

- A user visits the deployed frontend and opens the Recommendations page.

- The frontend sends a GET request to the backend recommendations endpoint.
- The backend queries the destinations table, calculates recommendation metrics, orders the results, and returns JSON.
- The frontend renders destination cards and optional wishlist controls.
- When an authenticated user creates or updates a wishlist, the frontend sends bearer-authenticated requests to protected endpoints on the backend.
- The MCP server uses the same deployed endpoints, which means MCP actions and website actions operate against the same environment.

3. API conventions and base information

Base URLs

- Local development base URL: http://127.0.0.1:8000
- Interactive documentation during local or deployed execution: /docs
- Deployed production base URL: the Railway service URL configured in the Vercel environment variable NEXT_PUBLIC_API_BASE_URL
- The API returns JSON for all operational endpoints.

General conventions

- Request and response payloads are JSON unless otherwise stated.
- Protected endpoints require an Authorization header using the Bearer scheme.
- Path parameters are used for resource identifiers such as destination_id, wishlist_id, and item_id.
- Query parameters are used for filtering and sorting recommendation results.
- The current implementation returns 200 OK for successful create operations; while some APIs use 201 Created for new resources, the examples in this document reflect the implemented behaviour.
- FastAPI validation failures are returned using the framework's default 422 Unprocessable Entity response model.

Headers commonly used by clients

```
Content-Type: application/json
Authorization: Bearer <ACCESS_TOKEN>
```

4. Authentication model and security behaviour

The API uses a bearer-token workflow for protected wishlist operations. A client first registers a user account and then authenticates using the login endpoint. The login response contains an access_token and token_type, and the access token must be sent in the Authorization header for subsequent requests to protected endpoints.

In the current coursework implementation, the access token is generated in the backend as a token string of the form demo-token-<user_id>. The backend validates that the Authorization header exists, starts with Bearer demo-token-, and contains a numeric user identifier. This is sufficient for a coursework demonstration of bearer-token protected access, although a production-grade system would typically replace this implementation with signed JWTs and stronger secret management.

Authentication workflow step by step

- Step 1 - Register a new account using POST /auth/register.
- Step 2 - Log in using POST /auth/login with the same email and password.

- Step 3 - Extract access_token from the login response.
- Step 4 - Send protected requests with the header Authorization: Bearer <access_token>.
- Step 5 - If the backend responds with 401 Unauthorized, the client should treat the session as invalid or expired and re-authenticate.

Authentication behaviour enforced by the backend

- Blank or whitespace-only email values are rejected.
- Blank or whitespace-only passwords are rejected.
- Passwords shorter than the implemented minimum length are rejected.
- Duplicate registration attempts using an existing email are rejected.
- Protected wishlist operations return 401 if the Authorization header is missing or malformed.
- Wishlist access is scoped per user; one user cannot access or delete another user's wishlists.

Example bearer header

```
Authorization: Bearer demo-token-3
```

5. Resource model summary

The API exposes four main resource groups: users, destinations, recommendations, and wishlists. The destinations resource is read-only and derived from a source dataset. Recommendations are generated dynamically from destinations and do not have their own persisted table. Wishlists and wishlist items are persisted per user and require authentication.

Below is a narrative description of the core entities returned by the API.

Destination object

- id - integer identifier for the destination record.
- name - destination name.
- country - country name.
- continent - continent label used for filtering.
- type - high-level destination type such as cultural, coastal, city, nature, or adventure.
- best_season - best season to visit.
- avg_cost_usd - indicative average travel cost in US dollars.
- rating - destination quality score from the dataset.
- annual_visitors_m - annual visitor count measured in millions.
- unesco - boolean-style value indicating UNESCO status.

Recommendation object

- Contains the destination fields above plus derived scoring fields.
- affordability_score - higher when the destination is cheaper.
- quietness_score - higher when the destination has lower visitor numbers.
- quality_score - derived from the rating value.
- hidden_gem_score - derived from rating and quietness.
- barrier_score - combined score used as the default ranking order.

Wishlist object

- id - integer identifier for the wishlist.
- user_id - identifier of the owning user.

- name - wishlist title.
- description - optional free-text description.
- created_at - timestamp assigned when the wishlist is created.

Wishlist item object

- id - integer identifier for the wishlist item row.
- wishlist_id - identifier of the owning wishlist.
- destination_id - destination identifier linked to the item.
- notes - optional free-text notes supplied by the user.
- priority - optional priority integer.
- created_at - timestamp assigned when the item is created.
- The implemented item responses also flatten selected destination metadata into the same JSON object so that the frontend can render destination details without making extra lookup calls.

6. Endpoint reference

6.1 Root endpoint

GET /

- Purpose: Health-style root endpoint used to confirm that the API process is running and to expose the database path configured by the application.
- Authentication: Public endpoint - no authentication required.
- Parameters:
  - No parameters.
- Example request:

```
curl -X GET "http://127.0.0.1:8000/"
```

- Example successful response:

```
{
  "message": "FastAPI is running",
  "database": "/path/to/travel.db"
}
```

- Possible error responses:
  - 500 Internal Server Error - unexpected application start-up or runtime error.
- Implementation notes:
  - Useful as a quick deployment smoke test after Railway or local startup.

6.2 Authentication endpoints

POST /auth/register

- Purpose: Create a new user account so the user can later authenticate and access protected wishlist endpoints.
- Authentication: Public endpoint - no authentication required.
- Request body:
  - email (string, required) - email address for the account. The backend trims whitespace and normalises the value to lowercase.

- password (string, required) - user password. Blank values are rejected and minimum-length validation is applied.
- Example request:

```
curl -X POST "http://127.0.0.1:8000/auth/register" \
  -H "Content-Type: application/json" \
  -d '{
    "email": "user@example.com",
    "password": "Password123"
  }'
```

- Example successful response:

```
{
  "message": "User registered successfully",
  "user_id": 7
}
```

- Possible error responses:
  - 400 Bad Request - An account with this email already exists.
  - 422 Unprocessable Entity - request body is missing fields, email is blank after trimming, or password fails validation.
  - 500 Internal Server Error - unexpected database error.
- Implementation notes:
  - Registration creates a database row in the users table.
  - At present, the API returns a confirmation message and numeric user_id rather than a full user object.

POST /auth/login

- Purpose: Authenticate an existing user and return a bearer-style access token required for protected requests.
- Authentication: Public endpoint - no authentication required.
- Request body:
  - email (string, required) - email address used during registration.
  - password (string, required) - password associated with the account.
- Example request:

```
curl -X POST "http://127.0.0.1:8000/auth/login" \
  -H "Content-Type: application/json" \
  -d '{
    "email": "user@example.com",
    "password": "Password123"
  }'
```

- Example successful response:

```
{
  "access_token": "demo-token-7",
  "token_type": "bearer",
  "user": {
    "id": 7,
    "email": "user@example.com"
  }
}
```

- Possible error responses:
  - 401 Unauthorized - invalid email or password.
  - 422 Unprocessable Entity - malformed or incomplete body, or blank values rejected by validation.
  - 500 Internal Server Error - unexpected database error.

- Implementation notes:
  - Clients should persist access_token and include it in the Authorization header for protected endpoints.
  - The returned token format is intentionally simple for coursework demonstration and is not a signed production JWT.

6.3 Destination endpoints

GET /destinations

- Purpose: Return a paginated slice of raw destination records from the imported dataset.
- Authentication: Public endpoint - no authentication required.
- Parameters:
  - limit (integer, optional, default 100) - maximum number of records to return; must be at least 1.
  - offset (integer, optional, default 0) - number of initial rows to skip.
- Example request:

```
curl -X GET "http://127.0.0.1:8000/destinations?limit=3&offset=0"
```

- Example successful response:

```
[
  {
    "id": 1,
    "name": "Kyoto",
    "country": "Japan",
    "continent": "Asia",
    "type": "Cultural",
    "best_season": "Spring",
    "avg_cost_usd": 280.0,
    "rating": 4.8,
    "annual_visitors_m": 2.1,
    "unesco": 1
  },
  {
    "id": 2,
    "name": "Porto",
    "country": "Portugal",
    "continent": "Europe",
    "type": "City",
    "best_season": "Autumn",
    "avg_cost_usd": 170.0,
    "rating": 4.6,
    "annual_visitors_m": 1.4,
    "unesco": 0
  }
]
```

- Possible error responses:
  - 422 Unprocessable Entity - invalid limit or offset value.
  - 500 Internal Server Error - unexpected database query failure.
- Implementation notes:
  - This endpoint is useful for raw dataset browsing and debugging.
  - It does not calculate derived recommendation metrics.

GET /destinations/count

- Purpose: Return the total number of imported destination records currently stored in the database.

- Authentication: Public endpoint - no authentication required.
- Parameters:
  - No parameters.
- Example request:

```
curl -X GET "http://127.0.0.1:8000/destinations/count"
```

- Example successful response:

```
{
  "count": 2000
}
```

- Possible error responses:
  - 500 Internal Server Error - unexpected database query failure.
- Implementation notes:
  - Useful for validating that the import pipeline has populated the database in the target environment.

GET /destinations/{destination_id}

- Purpose: Return a single destination record by identifier.
- Authentication: Public endpoint - no authentication required.
- Parameters:
  - destination_id (path integer, required) - identifier of the destination to retrieve.
- Example request:

```
curl -X GET "http://127.0.0.1:8000/destinations/12"
```

- Example successful response:

```
{
  "id": 12,
  "name": "Prague",
  "country": "Czech Republic",
  "continent": "Europe",
  "type": "Cultural",
  "best_season": "Spring",
  "avg_cost_usd": 190.0,
  "rating": 4.7,
  "annual_visitors_m": 3.0,
  "unesco": 1
}
```

- Possible error responses:
  - 404 Not Found - destination does not exist.
  - 422 Unprocessable Entity - destination_id is not a valid integer.
  - 500 Internal Server Error - unexpected database query failure.
- Implementation notes:
  - This endpoint is appropriate when a client already knows the destination identifier and needs a direct record lookup.

6.4 Recommendation endpoint

GET /recommendations

- Purpose: Return dynamically ranked destination recommendations based on cost, crowd levels, and quality. This is the most important public endpoint in the application.
- Authentication: Public endpoint - no authentication required.

- Parameters:
  - continent (string, optional) - restrict results to a specific continent.
  - country (string, optional) - partial country-name filter using a LIKE query.
  - min_rating (number, optional) - minimum permitted rating.
  - max_cost (number, optional) - maximum permitted average cost.
  - sort_by (string, optional) - custom sort strategy. Supported values are rating, cost_asc, cost_desc, visitors_asc, visitors_desc, name, affordability, quietness, quality, and hidden_gem. If omitted, results are sorted by barrier_score descending.
- Example request:

```
curl -X GET "http://127.0.0.1:8000/recommendations?
continent=Europe&max_cost=250&sort_by=affordability"
```

- Example successful response:

```
[
  {
    "id": 12,
    "name": "Prague",
    "country": "Czech Republic",
    "continent": "Europe",
    "type": "Cultural",
    "best_season": "Spring",
    "avg_cost_usd": 190.0,
    "rating": 4.7,
    "annual_visitors_m": 3.0,
    "unesco": 1,
    "affordability_score": 5.24,
    "quietness_score": 2.5,
    "quality_score": 4.7,
    "hidden_gem_score": 7.2,
    "barrier_score": 12.44
  }
]
```

- Possible error responses:
  - 422 Unprocessable Entity - invalid query parameter types, for example non-numeric min_rating or max_cost.
  - 500 Internal Server Error - unexpected database query or calculation error.
- Implementation notes:
  - The recommendation engine calculates derived scores at request time rather than storing them persistently in the database.
  - The default ordering uses barrier_score descending, which favours cheaper, quieter, better-rated destinations.

6.5 Wishlist endpoints

GET /wishlists

- Purpose: Return all wishlists owned by the authenticated user.
- Authentication: Protected endpoint - bearer token required.
- Parameters:
  - No query parameters.
- Example request:

```
curl -X GET "http://127.0.0.1:8000/wishlists" \
  -H "Authorization: Bearer demo-token-7"
```

- Example successful response:

```
[
  {
    "id": 5,
    "user_id": 7,
    "name": "My Dream Destinations",
    "description": "Places I would love to visit",
    "created_at": "2026-03-08 00:45:16"
  }
]
```

- Possible error responses:
  - 401 Unauthorized - missing Authorization header or invalid token format.
  - 500 Internal Server Error - unexpected database failure.
- Implementation notes:
  - Results are scoped to the authenticated user only.

POST /wishlists

- Purpose: Create a new wishlist owned by the authenticated user.
- Authentication: Protected endpoint - bearer token required.
- Request body:
  - name (string, required) - wishlist title; blank values are rejected.
  - description (string, optional) - supporting description text.
- Example request:

```
curl -X POST "http://127.0.0.1:8000/wishlists" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer demo-token-7" \
  -d '{
    "name": "My Dream Destinations",
    "description": "Places I would love to visit"
  }'
```

- Example successful response:

```
{
  "id": 5,
  "user_id": 7,
  "name": "My Dream Destinations",
  "description": "Places I would love to visit",
  "created_at": "2026-03-08 00:45:16"
}
```

- Possible error responses:
  - 401 Unauthorized - missing or invalid bearer token.
  - 422 Unprocessable Entity - body missing required fields or name is blank after validation.
  - 500 Internal Server Error - database write failure.
- Implementation notes:
  - The current implementation explicitly inserts CURRENT_TIMESTAMP when the row is created.
  - Clients typically refresh the wishlist list after creation.

GET /wishlists/{wishlist_id}

- Purpose: Return a single wishlist owned by the authenticated user.
- Authentication: Protected endpoint - bearer token required.

- Parameters:
  - wishlist_id (path integer, required) - identifier of the wishlist.
- Example request:

```
curl -X GET "http://127.0.0.1:8000/wishlists/5" \
  -H "Authorization: Bearer demo-token-7"
```

- Example successful response:

```
{
  "id": 5,
  "user_id": 7,
  "name": "My Dream Destinations",
  "description": "Places I would love to visit",
  "created_at": "2026-03-08 00:45:16"
}
```

- Possible error responses:
  - 401 Unauthorized - missing or invalid bearer token.
  - 404 Not Found - wishlist does not exist or does not belong to the authenticated user.
  - 422 Unprocessable Entity - invalid wishlist_id type.
- Implementation notes:
  - This endpoint returns the wishlist metadata only; items are retrieved from the dedicated items endpoint.

PATCH /wishlists/{wishlist_id}

- Purpose: Update the name and/or description of an existing wishlist owned by the authenticated user.
- Authentication: Protected endpoint - bearer token required.
- Parameters:
  - wishlist_id (path integer, required) - identifier of the wishlist to update.
- Request body:
  - name (string, optional) - replacement wishlist name.
  - description (string, optional) - replacement description.
- Example request:

```
curl -X PATCH "http://127.0.0.1:8000/wishlists/5" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer demo-token-7" \
  -d '{
    "name": "Updated Europe Wishlist",
    "description": "Revised shortlist"
  }'
```

- Example successful response:

```
{
  "id": 5,
  "user_id": 7,
  "name": "Updated Europe Wishlist",
  "description": "Revised shortlist",
  "created_at": "2026-03-08 00:45:16"
}
```

- Possible error responses:
  - 401 Unauthorized - missing or invalid bearer token.
  - 404 Not Found - wishlist does not exist or is not owned by the user.
  - 422 Unprocessable Entity - invalid JSON or invalid wishlist_id.
- Implementation notes:

- If a field is omitted, the backend keeps the previous stored value.

DELETE /wishlists/{wishlist_id}

- Purpose: Delete an entire wishlist and all of its associated wishlist items.
- Authentication: Protected endpoint - bearer token required.
- Parameters:
  - wishlist_id (path integer, required) - identifier of the wishlist to delete.
- Example request:

```
curl -X DELETE "http://127.0.0.1:8000/wishlists/5" \
  -H "Authorization: Bearer demo-token-7"
```

- Example successful response:

```
{
  "message": "Wishlist deleted successfully"
}
```

- Possible error responses:
  - 401 Unauthorized - missing or invalid bearer token.
  - 404 Not Found - wishlist does not exist or does not belong to the authenticated user.
  - 422 Unprocessable Entity - invalid wishlist_id.
- Implementation notes:
  - The backend deletes associated wishlist_items before removing the parent wishlist row.
  - After deletion, clients should remove the deleted list from local UI state and, if necessary, load another available wishlist.

POST /wishlists/{wishlist_id}/items

- Purpose: Add a destination to a wishlist owned by the authenticated user.
- Authentication: Protected endpoint - bearer token required.
- Parameters:
  - wishlist_id (path integer, required) - target wishlist identifier.
- Request body:
  - destination_id (integer, required) - identifier of the destination to save.
  - notes (string, optional) - user notes associated with the item.
  - priority (integer, optional) - optional priority value.
- Example request:

```
curl -X POST "http://127.0.0.1:8000/wishlists/5/items" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer demo-token-7" \
  -d '{
    "destination_id": 12,
    "notes": "High priority spring trip",
    "priority": 1
  }'
```

- Example successful response:

```
{
  "id": 18,
  "wishlist_id": 5,
  "destination_id": 12,
  "notes": "High priority spring trip",
  "priority": 1,
```

```
    "created_at": "2026-03-08 01:03:14",
    "destination_inner_id": 12,
    "name": "Prague",
    "country": "Czech Republic",
    "continent": "Europe",
    "type": "Cultural",
    "best_season": "Spring",
    "avg_cost_usd": 190.0,
    "rating": 4.7,
    "annual_visitors_m": 3.0,
    "unesco": 1
  }
```

- Possible error responses:
  - 401 Unauthorized - missing or invalid bearer token.
  - 404 Not Found - wishlist does not exist or destination does not exist.
  - 400 Bad Request - destination already in wishlist.
  - 422 Unprocessable Entity - invalid body or invalid numeric types.
- Implementation notes:
  - This endpoint joins destination metadata into the returned item object so the frontend can render the saved destination immediately.
  - Duplicate destination entries in the same wishlist are explicitly prevented.

GET /wishlists/{wishlist_id}/items

- Purpose: Return all items stored in a specific wishlist owned by the authenticated user.
- Authentication: Protected endpoint - bearer token required.
- Parameters:
  - wishlist_id (path integer, required) - identifier of the wishlist whose items should be listed.
- Example request:

```
curl -X GET "http://127.0.0.1:8000/wishlists/5/items" \
  -H "Authorization: Bearer demo-token-7"
```

- Example successful response:

```
[
  {
    "id": 18,
    "wishlist_id": 5,
    "destination_id": 12,
    "notes": "High priority spring trip",
    "priority": 1,
    "created_at": "2026-03-08 01:03:14",
    "destination_inner_id": 12,
    "name": "Prague",
    "country": "Czech Republic",
    "continent": "Europe",
    "type": "Cultural",
    "best_season": "Spring",
    "avg_cost_usd": 190.0,
    "rating": 4.7,
    "annual_visitors_m": 3.0,
    "unesco": 1
  }
]
```

- Possible error responses:
  - 401 Unauthorized - missing or invalid bearer token.
  - 404 Not Found - wishlist does not exist or is not owned by the user.

    - 422 Unprocessable Entity - invalid wishlist_id.
- Implementation notes:
    - This is the primary endpoint used by the frontend to populate the Saved Destinations section.

GET /wishlists/{wishlist_id}/items/{item_id}

- Purpose: Return a single wishlist item with flattened destination metadata.
- Authentication: Protected endpoint - bearer token required.
- Parameters:
    - wishlist_id (path integer, required) - parent wishlist identifier.
    - item_id (path integer, required) - wishlist item identifier.
- Example request:

```
curl -X GET "http://127.0.0.1:8000/wishlists/5/items/18" \
  -H "Authorization: Bearer demo-token-7"
```

- Example successful response:

```
{
  "id": 18,
  "wishlist_id": 5,
  "destination_id": 12,
  "notes": "High priority spring trip",
  "priority": 1,
  "created_at": "2026-03-08 01:03:14",
  "destination_inner_id": 12,
  "name": "Prague",
  "country": "Czech Republic",
  "continent": "Europe",
  "type": "Cultural",
  "best_season": "Spring",
  "avg_cost_usd": 190.0,
  "rating": 4.7,
  "annual_visitors_m": 3.0,
  "unesco": 1
}
```

- Possible error responses:
    - 401 Unauthorized - missing or invalid bearer token.
    - 404 Not Found - wishlist item not found, or parent wishlist not found/owned by user.
    - 422 Unprocessable Entity - invalid identifier types.
- Implementation notes:
    - Useful for item-specific debugging or future expansion to detailed item views.

PATCH /wishlists/{wishlist_id}/items/{item_id}

- Purpose: Update the notes and/or priority stored against a specific wishlist item.
- Authentication: Protected endpoint - bearer token required.
- Parameters:
    - wishlist_id (path integer, required) - parent wishlist identifier.
    - item_id (path integer, required) - item identifier.
- Request body:
    - notes (string, optional) - replacement notes text.
    - priority (integer, optional) - replacement priority.
- Example request:

```
curl -X PATCH "http://127.0.0.1:8000/wishlists/5/items/18" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer demo-token-7" \
  -d '{
    "notes": "Updated note after more research",
    "priority": 2
  }'
```

- Example successful response:

```
{
  "id": 18,
  "wishlist_id": 5,
  "destination_id": 12,
  "notes": "Updated note after more research",
  "priority": 2,
  "created_at": "2026-03-08 01:03:14",
  "destination_inner_id": 12,
  "name": "Prague",
  "country": "Czech Republic",
  "continent": "Europe",
  "type": "Cultural",
  "best_season": "Spring",
  "avg_cost_usd": 190.0,
  "rating": 4.7,
  "annual_visitors_m": 3.0,
  "unesco": 1
}
```

- Possible error responses:
  - 401 Unauthorized - missing or invalid bearer token.
  - 404 Not Found - wishlist not found or item not found.
  - 422 Unprocessable Entity - malformed body or invalid numeric types.
- Implementation notes:
  - Only notes and priority are currently mutable at item level.

DELETE /wishlists/{wishlist_id}/items/{item_id}

- Purpose: Delete a single item from a wishlist without deleting the entire wishlist.
- Authentication: Protected endpoint - bearer token required.
- Parameters:
  - wishlist_id (path integer, required) - parent wishlist identifier.
  - item_id (path integer, required) - item identifier to remove.
- Example request:

```
curl -X DELETE "http://127.0.0.1:8000/wishlists/5/items/18" \
  -H "Authorization: Bearer demo-token-7"
```

- Example successful response:

```
{
  "message": "Wishlist item deleted successfully"
}
```

- Possible error responses:
  - 401 Unauthorized - missing or invalid bearer token.
  - 404 Not Found - wishlist not found.
  - 422 Unprocessable Entity - invalid identifier types.
- Implementation notes:
  - This endpoint is used by the frontend's Remove from Wishlist button.

7. Error handling and status codes

The API uses standard HTTP status codes and FastAPI's validation model to communicate success and failure. Clients should always examine the response status code before assuming a body shape. In some cases the backend returns a simple JSON object containing a message or detail field; in validation cases FastAPI returns an array of error descriptors under detail.

The list below summarises the most important status codes used across the system.

- 200 OK - successful request, including successful create, update, and delete operations in the current implementation.
- 400 Bad Request - semantic problem with the request, for example duplicate destination insertion into a wishlist or duplicate user registration.
- 401 Unauthorized - missing, invalid, or unusable bearer token, or invalid login credentials.
- 404 Not Found - requested resource does not exist or is outside the authenticated user's scope.
- 422 Unprocessable Entity - FastAPI or Pydantic request validation failed.
- 500 Internal Server Error - unexpected runtime, database, or deployment issue.

Representative error examples

- Duplicate registration example

```
{
  "detail": "An account with this email already exists."
}
```

- Invalid login example

```
{
  "detail": "Invalid email or password"
}
```

- Invalid token example

```
{
  "detail": "Invalid token"
}
```

- FastAPI validation example

```
{
  "detail": [
    {
      "loc": ["body", "password"],
      "msg": "Value error, Password cannot be blank",
      "type": "value_error"
    }
  ]
}
```

- Duplicate wishlist item example

```
{
  "detail": "Destination already in wishlist"
}
```

8. End-to-end usage workflows

Workflow A - discover destinations anonymously

- Call GET /recommendations without authentication.

- Optionally add filters such as continent, country, min_rating, max_cost, or sort_by.
- Render the returned destination objects and derived scores in the client interface.

Workflow B - register, log in, and create a wishlist

- Register a new account with POST /auth/register.
- Authenticate with POST /auth/login and store access_token from the response.
- Create a wishlist with POST /wishlists using the bearer token.
- Refresh GET /wishlists to show the new list.
- Add destinations using POST /wishlists/{wishlist_id}/items.

Workflow C - save and remove destinations

- Use GET /recommendations to choose a destination.
- Submit POST /wishlists/{wishlist_id}/items with destination_id.
- Use GET /wishlists/{wishlist_id}/items to populate the saved items view.
- Remove an item with DELETE /wishlists/{wishlist_id}/items/{item_id} when it is no longer relevant.

9. Deployment, environments, and operational notes

The system has been deployed as a split frontend-backend architecture. The Next.js frontend is hosted on Vercel, while the FastAPI backend is hosted on Railway. The frontend reads the deployed backend URL from the environment variable NEXT_PUBLIC_API_BASE_URL, and the backend is configured with CORS rules to allow requests from the deployed Vercel origin and local development origins.

This deployment arrangement demonstrates a realistic separation of presentation and service layers. It also introduces practical concerns such as environment-variable management, origin control, and the importance of ensuring that the frontend and backend both target the same live environment.

The project currently uses SQLite as its backing store. This is effective for coursework prototyping and lightweight deployments, but it is important to note that some hosting environments treat local filesystem state as ephemeral. For a production-grade system, a managed relational database such as PostgreSQL would normally be preferred for stronger persistence, concurrency handling, and operational durability.

Operational notes

- The backend should be smoke-tested after deployment by opening /docs and /recommendations.
- The frontend should be rebuilt or redeployed whenever the production API base URL changes.
- Authentication issues in production are often traceable to stale localStorage tokens or CORS configuration.
- If recommendations are empty in a fresh environment, the first check should be whether the destinations dataset was imported into the deployed database.

10. MCP integration appendix

In Phase 3 of the project, the API was extended with an MCP server. The MCP layer does not replace the REST API; instead, it provides an AI-callable interface over the deployed backend. This means an MCP-compatible client can retrieve recommendations, register and log in users, create wishlists, and list wishlists using structured tool calls.

The first local prototype of the MCP server interacted directly with a local SQLite database. This created an environment mismatch, because actions performed through MCP did not appear on the deployed website. The implementation was therefore improved so that the MCP server now calls the deployed

Railway backend over HTTP. As a result, MCP actions and web actions now operate on the same live system.

MCP tools currently exposed

- get_recommendations
- register_user
- login_user
- list_wishlists
- create_wishlist
- delete_wishlist
- plan_low_stress_trip prompt

Why MCP matters for this project

- It demonstrates creative use of generative AI beyond prompt drafting alone.
- It shows that the backend can be consumed not only by web clients but also by AI tool ecosystems.
- It strengthens the project architecture by making the service AI-accessible without changing the frontend or core API design.

11. Local development and testing notes

The project was developed and validated locally before deployment. Typical setup includes creating a virtual environment, installing dependencies, importing the source destination dataset, and running the FastAPI application with Uvicorn. During development, the interactive Swagger UI at /docs was used for request inspection and debugging.

A structured testing approach was also applied. Core tests and manual validation covered registration, login, recommendations retrieval, wishlist CRUD behaviour, invalid-token handling, duplicate-user prevention, blank-input rejection, and MCP-triggered updates against the live environment.

Typical local startup sequence

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python scripts/import_destinations.py
uvicorn app.main:app --reload --port 8000
```

Suggested smoke tests

- Open /docs and verify that all expected routes are present.
- Open /recommendations and confirm the response is non-empty.
- Register a new user, log in, create a wishlist, and add a destination item.
- Run the MCP Inspector against mcp_server.py and confirm that a wishlist created through MCP appears on the deployed website.

Conclusion

The Travel Without Barriers API combines a non-trivial data model, dynamic recommendation logic, authentication, wishlist management, cloud deployment, and AI-facing MCP integration in one coherent system. Its primary strength lies in translating a real user problem - reducing travel-planning barriers - into a practical service-oriented architecture with clear, reusable interfaces.

This documentation has been structured to ensure that a reader can understand both the behaviour of each endpoint and the broader capabilities of the platform. A developer reading this document should be able to authenticate, request recommendations, manage wishlists, interpret returned payloads, and understand how the API fits into both the web application and the MCP integration layer.

Appendix A - Quick reference checklist

- Public data discovery begins with GET /recommendations and GET /destinations.
- User onboarding begins with POST /auth/register followed by POST /auth/login.
- All wishlist operations require Authorization: Bearer <access_token>.
- Wishlist metadata is managed with GET, POST, PATCH, and DELETE on /wishlists and /wishlists/{wishlist_id}.
- Saved destination records are managed with POST, GET, PATCH, and DELETE on the /wishlists/{wishlist_id}/items routes.
- The deployed frontend and the MCP layer both depend on the same Railway backend, ensuring consistent live behaviour across web and AI-driven interactions.
- Recommended smoke-test sequence: open /docs, confirm /recommendations is populated, register a user, log in, create a wishlist, add a destination, and verify the saved item appears in the UI.